

TP2 – Optimisation des Réseaux Feedforward Profonds

Hyperparamètres, régularisation et stabilisation de l'apprentissage

Réalisé par : **Hatim ELAALGI** et **Hamza ELAZAAR**

Master AI Digital Computing

Encadré par : **Monsieur Youssef SAADI**

Année universitaire 2025–2026

Résumé

Ce rapport présente la réalisation complète du TP portant sur l'optimisation des réseaux feedforward profonds appliqués au jeu de données California Housing. L'objectif est de construire un modèle MLP en PyTorch, d'étudier l'effet de Batch Normalization, Dropout, régularisation L2 et gradient clipping, puis de comparer Grid Search et Random Search pour la recherche d'hyperparamètres. Les expérimentations ont été exécutées avec CUDA sur une carte NVIDIA GeForce GTX 1650 Ti. Le meilleur modèle final est issu du Random Search et obtient sur le jeu de test un MSE de 0.2509, un MAE de 0.3377 et un R^2 de 0.8105.

Table des matières

1	Introduction	2
2	Environnement expérimental	2
3	Exercice 1 – Exploration et préparation du dataset	2
3.1	Chargement et analyse exploratoire	2
3.2	Prétraitement et pipeline PyTorch	3
4	Exercice 2 – Modèle MLP et boucle d'entraînement	4
4.1	Architecture DeepFFN	4
4.2	Gradient clipping et entraînement	4
5	Exercice 3 – Analyse des techniques de stabilisation	5
5.1	Ablation study	5
5.2	Effet du gradient clipping	7
6	Exercice 4 – Grid Search	9
6.1	Définition de la grille	9
6.2	Meilleures configurations	9
6.3	Impact des hyperparamètres	11

7 Exercice 5 – Random Search	12
7.1 Espace de recherche	12
7.2 Résultats obtenus	12
7.3 Corrélations de Spearman	14
8 Exercice 6 – Évaluation finale	14
8.1 Validation sur le jeu de test	14
8.2 Analyse des erreurs	15
9 Synthèse et discussion	17
10 Conclusion	17

1 Introduction

Les réseaux feedforward profonds sont sensibles au choix des hyperparamètres, à l'initialisation des poids et aux techniques de régularisation. Dans ce TP, nous étudions ces aspects sur une tâche de régression : prédire la valeur médiane des maisons du dataset California Housing à partir de huit variables descriptives. La cible `MedHouseVal` est exprimée en centaines de milliers de dollars.

L'implémentation a été réalisée en PyTorch, avec un code volontairement simple et modulaire afin de rester adapté à un niveau Master. Le fichier `common_tp.py` contient le modèle `DeepFFN`, le pipeline de données, les boucles d'entraînement, l'évaluation et les fonctions de recherche d'hyperparamètres. Les fichiers `ex1_dataset.py` à `ex6_evaluation.py` correspondent aux exercices demandés.

2 Environnement expérimental

Les expériences ont été lancées avec les bibliothèques `torch`, `numpy`, `pandas`, `scikit-learn`, `matplotlib` et `seaborn`. Le calcul GPU a été activé avec CUDA. La carte graphique utilisée est une NVIDIA GeForce GTX 1650 Ti.

Pour assurer la reproductibilité, les graines aléatoires ont été fixées avec :

```
torch.manual_seed(42),  numpy.random.seed(42),  random.seed(42).
```

3 Exercice 1 – Exploration et préparation du dataset

3.1 Chargement et analyse exploratoire

Le dataset California Housing contient 20640 exemples et 8 variables explicatives : `MedInc`, `HouseAge`, `AveRooms`, `AveBedrms`, `Population`, `AveOccup`, `Latitude` et `Longitude`. La variable cible est `MedHouseVal`.

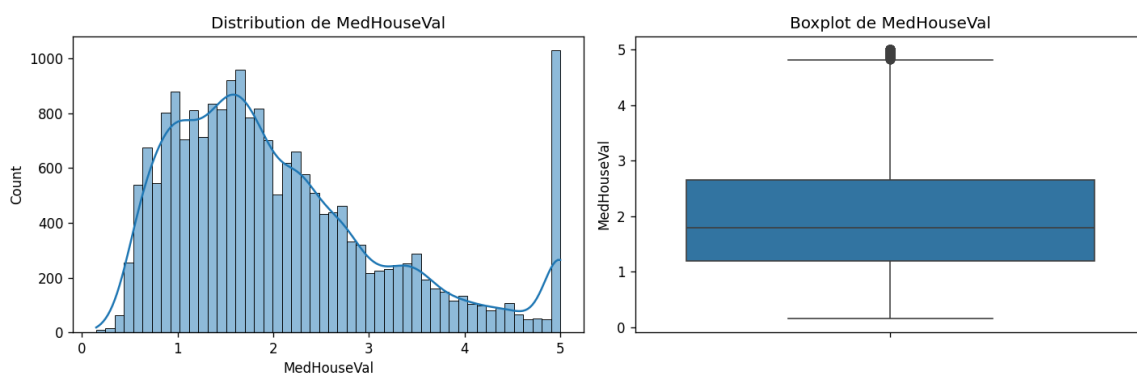


FIGURE 1 – Histogramme et boxplot de la variable cible `MedHouseVal`.

La distribution de la cible est asymétrique vers la droite. On observe également un plafonnement artificiel autour de 5.0, ce qui indique que certaines valeurs ont été censurées dans le dataset. Le boxplot confirme la présence de valeurs élevées proches de cette limite supérieure.

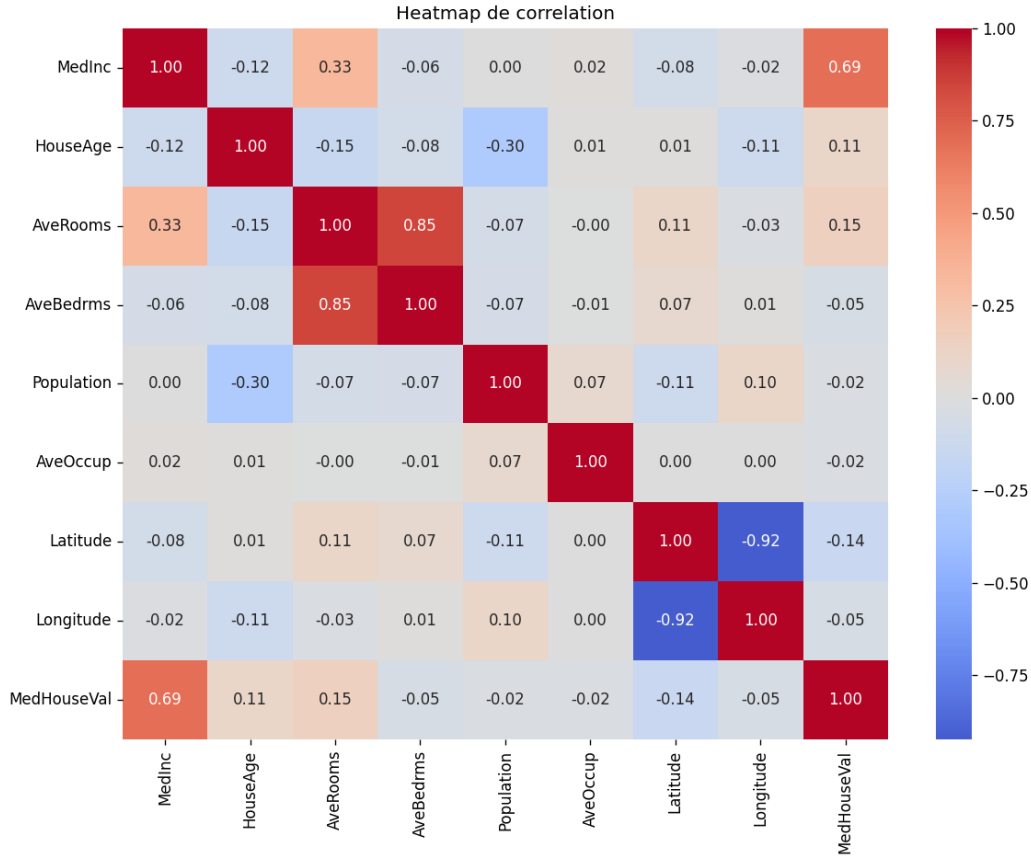


FIGURE 2 – Matrice de corrélation entre les variables du dataset.

La variable la plus corrélée à la cible est **MedInc**, avec une corrélation absolue d'environ 0.688. Cela signifie que le revenu médian du bloc est l'indicateur linéaire le plus informatif pour prédire le prix médian des maisons. La plus forte colinéarité entre variables explicatives est observée entre **Latitude** et **Longitude**, ce qui est logique car ces deux variables décrivent conjointement la position géographique.

3.2 Prétraitement et pipeline PyTorch

Le split demandé a été respecté :

70% train, 15% validation, 15% test.

La stratification a été approchée par discrétisation de la cible avec `pd.cut`. Le **StandardScaler** a été ajusté uniquement sur le jeu d'entraînement puis appliqué aux jeux de validation et de test. Cette règle est essentielle pour éviter le *data leakage*. Si le scaler était ajusté sur validation ou test, les statistiques de ces jeux influenceraient indirectement l'apprentissage.

Les **DataLoader** ont ensuite été créés avec les tailles de batch demandées : 64 pour l'entraînement et 256 pour validation/test. Les dimensions vérifiées sont conformes : $X \in \mathbb{R}^{64 \times 8}$ et $y \in \mathbb{R}^{64 \times 1}$.

4 Exercice 2 – Modèle MLP et boucle d’entraînement

4.1 Architecture DeepFFN

La classe `DeepFFN` implémente un MLP configurable. Les paramètres principaux sont : la liste des dimensions cachées, l’activation, l’utilisation ou non de `BatchNorm`, le taux de `Dropout` et la dimension de sortie. Les activations supportées sont `relu`, `tanh`, `leaky_relu`, `elu` et `selu`.

L’initialisation suit les consignes :

- initialisation de He avec `kaiming_normal_` pour `ReLU`, `LeakyReLU` et `ELU` ;
- initialisation Xavier uniforme pour les autres activations ;
- biais initialisés à zéro.

La configuration par défaut est :

`[128, 64, 32]`, `ReLU`, `BatchNorm` activé, `Dropout = 0.2`.

Elle contient 11969 paramètres entraînables.

4.2 Gradient clipping et entraînement

La fonction `train_one_epoch` applique le gradient clipping avant `optimizer.step()` à l’aide de :

```
nn.utils.clip_grad_norm_(model.parameters(), clip_value).
```

La norme du gradient avant clipping est enregistrée pour le suivi. La fonction `evaluate` calcule MSE, MAE et R^2 en mode `eval()` et sans gradient.

La boucle `train_model` utilise l’optimiseur Adam, le scheduler `ReduceLROnPlateau`, un early stopping et sauvegarde le meilleur modèle.

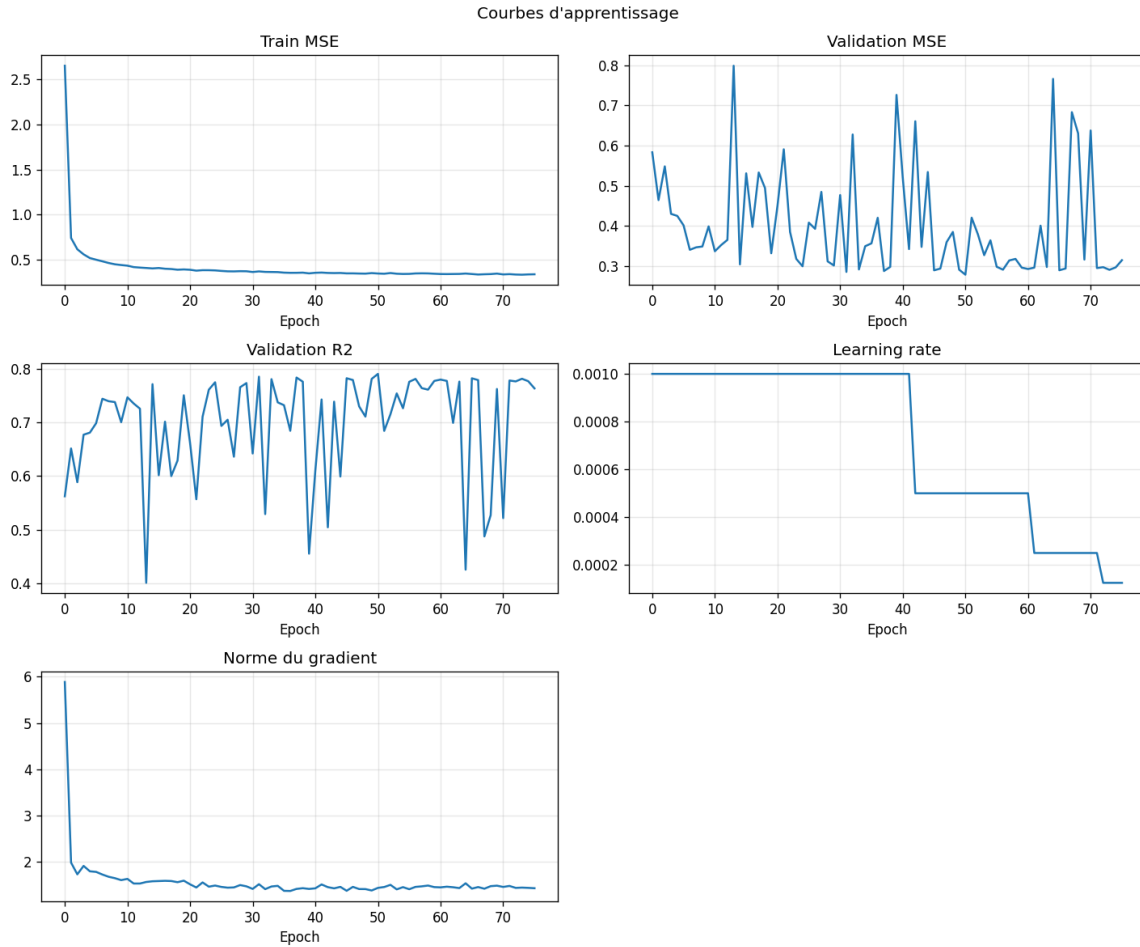


FIGURE 3 – Courbes d'apprentissage de la configuration baseline.

La baseline atteint une meilleure MSE de validation de 0.2790 lors de l'entraînement initial, avec un arrêt anticipé à l'epoch 76 et un temps d'entraînement de 95.9 s. Les courbes montrent une amélioration rapide au début, puis une stabilisation. La réduction du learning rate contribue à affiner l'optimisation lorsque la validation cesse de progresser.

5 Exercice 3 – Analyse des techniques de stabilisation

5.1 Ablation study

TABLE 1 – Résultats de l'étude d'ablation.

Configuration	Best Val MSE	Temps (s)
A – Baseline complet	0.3796	29.9
B – Sans BatchNorm	0.2596	97.3
C – Sans Dropout	0.3297	33.6
D – Sans L2	0.3555	50.3
E – Aucune régularisation	0.2687	48.1

Dans cette exécution, les configurations sans BatchNorm et sans régularisation obtiennent une meilleure MSE de validation que la baseline complète. Ce résultat ne signifie pas que la régularisation est inutile, mais indique que la baseline peut être légèrement trop régularisée pour ce split et cette architecture. Le Dropout à 0.2 peut notamment ralentir l'apprentissage et augmenter le biais du modèle.

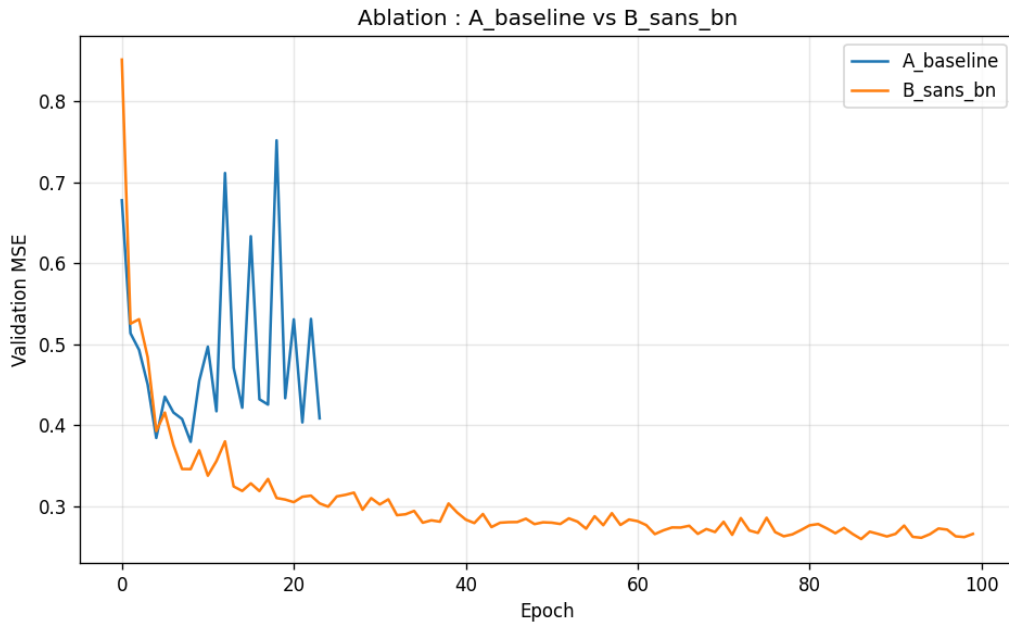


FIGURE 4 – Comparaison validation MSE : baseline vs sans BatchNorm.

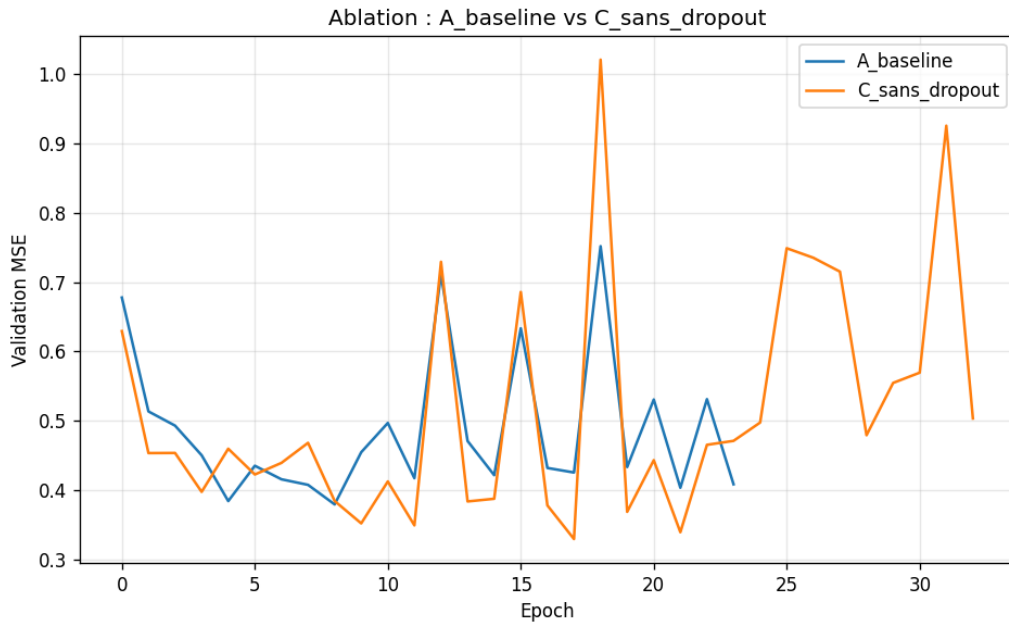


FIGURE 5 – Comparaison validation MSE : baseline vs sans Dropout.

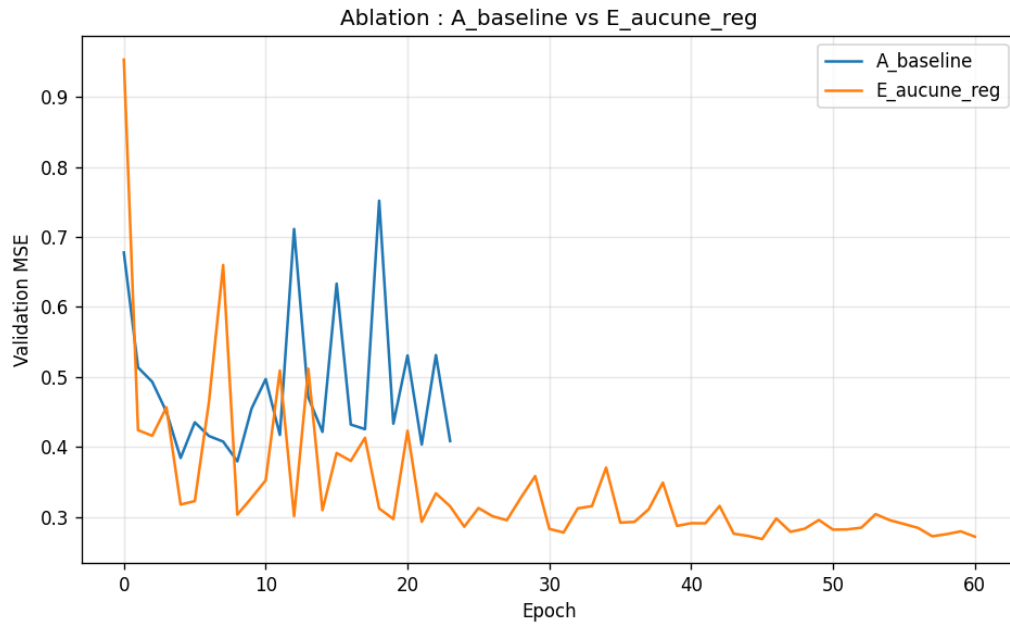


FIGURE 6 – Comparaison validation MSE : baseline vs aucune régularisation.

Les courbes permettent d’observer que BatchNorm n’est pas toujours indispensable sur ce dataset tabulaire de taille moyenne, mais elle reste une technique importante pour stabiliser des réseaux plus profonds ou des learning rates plus élevés. Dropout et L2 jouent davantage un rôle de contrôle du surapprentissage.

5.2 Effet du gradient clipping

Les valeurs testées sont : sans clipping, 0.1, 0.5, 1.0, 5.0 et 10.0.

TABLE 2 – Effet du gradient clipping sur la MSE de validation.

Configuration	Val MSE	Temps (s)
Sans clipping	0.3397	70.2
Clip 0.1	0.3402	31.7
Clip 0.5	0.3685	26.4
Clip 1.0	0.3796	34.9
Clip 5.0	0.3039	68.3
Clip 10.0	0.3161	58.5
Données brutes sans clipping	0.3770	71.5
Données brutes avec clip 1.0	0.4228	55.3

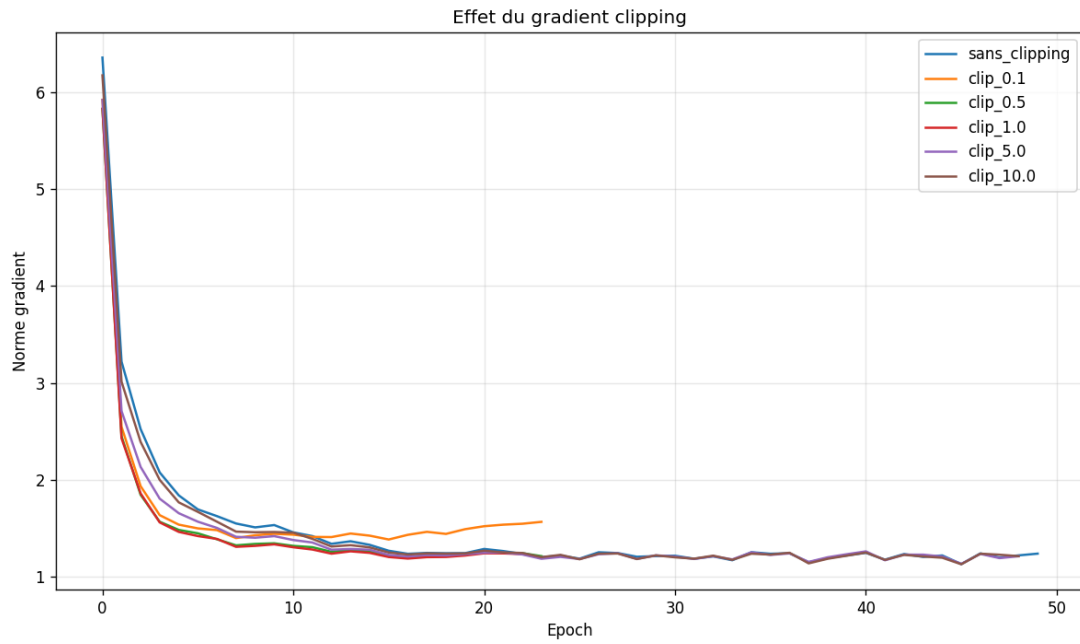


FIGURE 7 – Évolution de la norme du gradient selon la valeur de clipping.

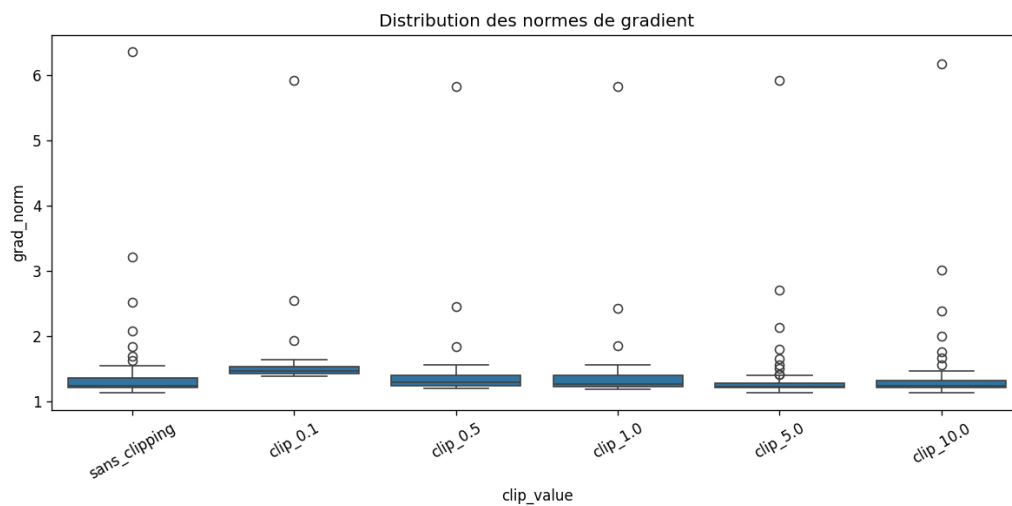


FIGURE 8 – Distribution des normes de gradient selon le clipping.

Un `clip_value` trop faible, comme 0.1, bride fortement les mises à jour et peut conduire à du sous-apprentissage. Une valeur trop grande, comme 10.0, protège peu contre les pics. Les valeurs autour de 1.0 à 5.0 constituent un compromis raisonnable.

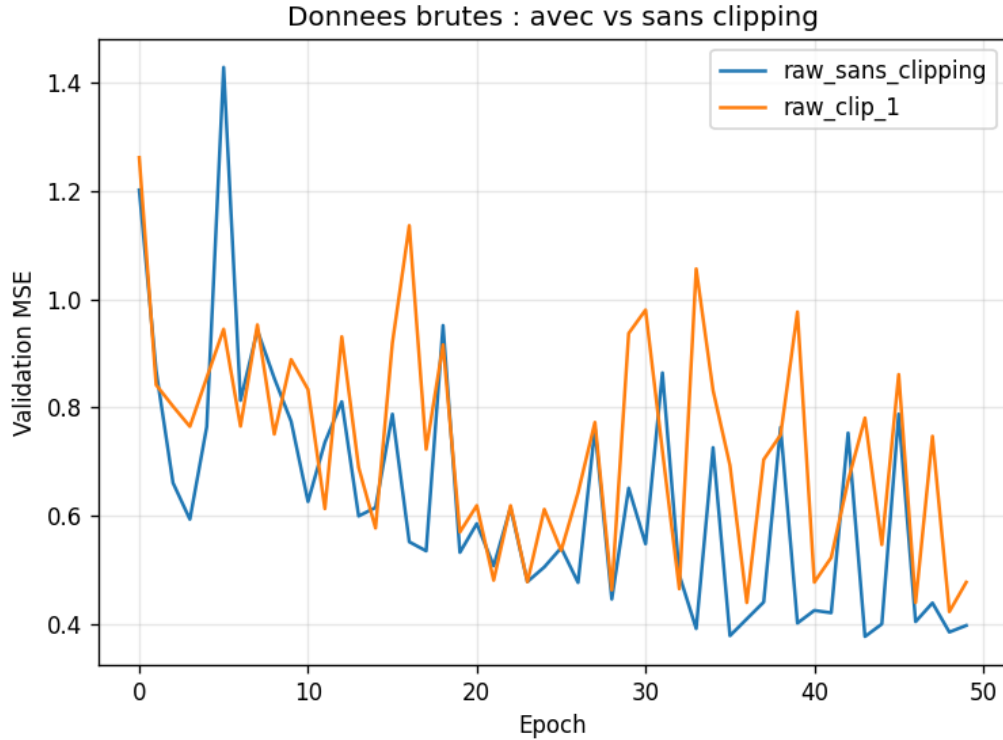


FIGURE 9 – Comparaison avec et sans gradient clipping sur données non standardisées.

Sur les données brutes non standardisées, l’instabilité augmente nettement. Le gradient clipping joue alors un rôle protecteur en limitant les explosions de gradient. Cette expérience confirme aussi l’importance du `StandardScaler`.

6 Exercice 4 – Grid Search

6.1 Définition de la grille

La grille complète contient :

$$4 \times 3 \times 3 \times 2 \times 2 \times 2 = 288$$

configurations. Pour garder un temps raisonnable, le TP demande une grille réduite de 48 configurations. Celle-ci a été exécutée avec 80 epochs maximum par configuration.

6.2 Meilleures configurations

TABLE 3 – Top 5 des configurations Grid Search.

Architecture	Activation	Dropout	LR	Weight decay	Val MSE
[128,64,32]	leaky_relu	0.3	0.0005	0.0010	0.2883
[128,64,32]	relu	0.3	0.0005	0.0010	0.2895
[128,64,32]	relu	0.3	0.0005	0.0001	0.2981
[128,64,32]	leaky_relu	0.3	0.0005	0.0001	0.2986
[128,64,32]	relu	0.3	0.0010	0.0001	0.3012

La meilleure configuration Grid Search utilise l'architecture [128, 64, 32], l'activation LeakyReLU, un Dropout de 0.3, un learning rate de 0.0005 et un weight decay de 0.001.

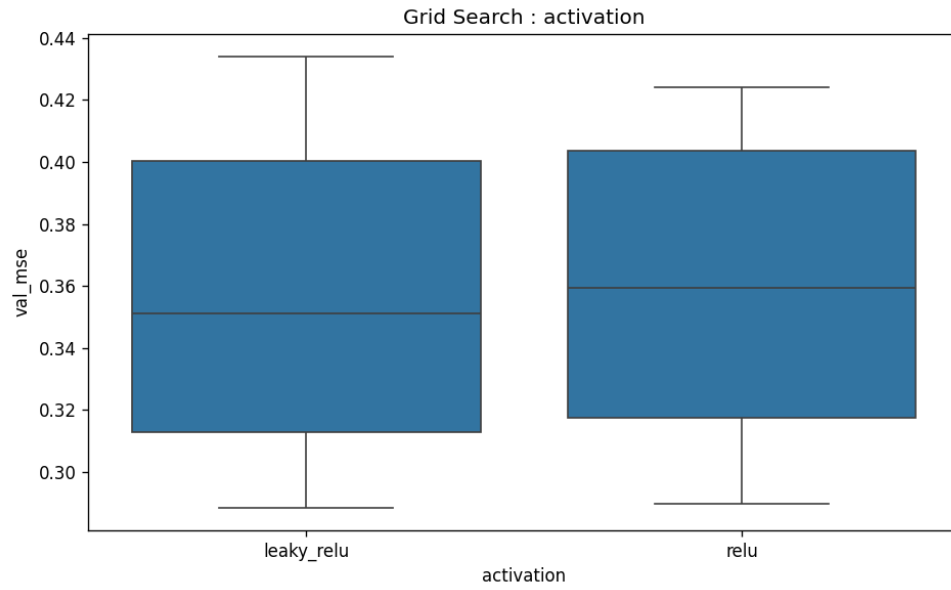


FIGURE 10 – Influence du choix de l'activation sur la MSE de validation.

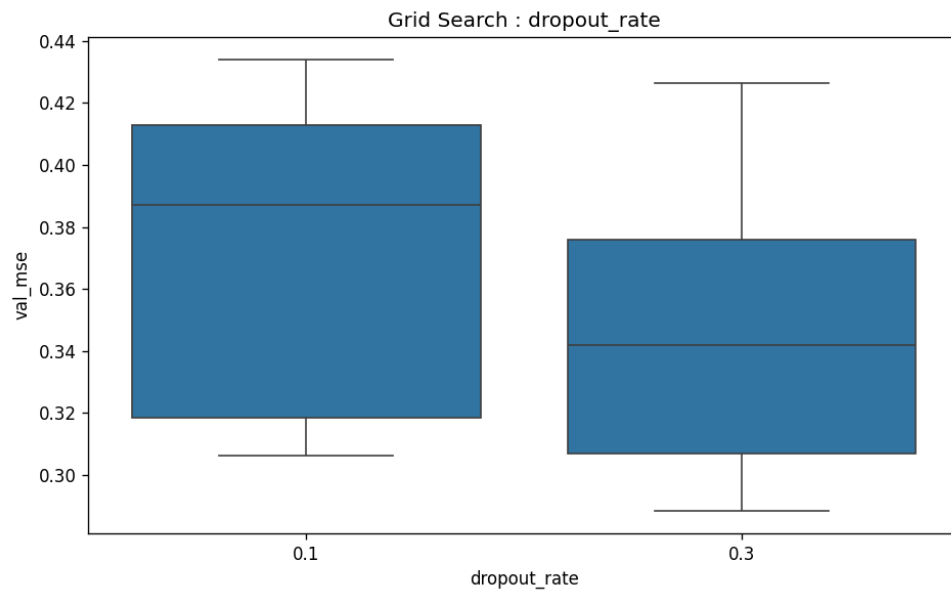


FIGURE 11 – Influence du taux de Dropout sur la MSE de validation.

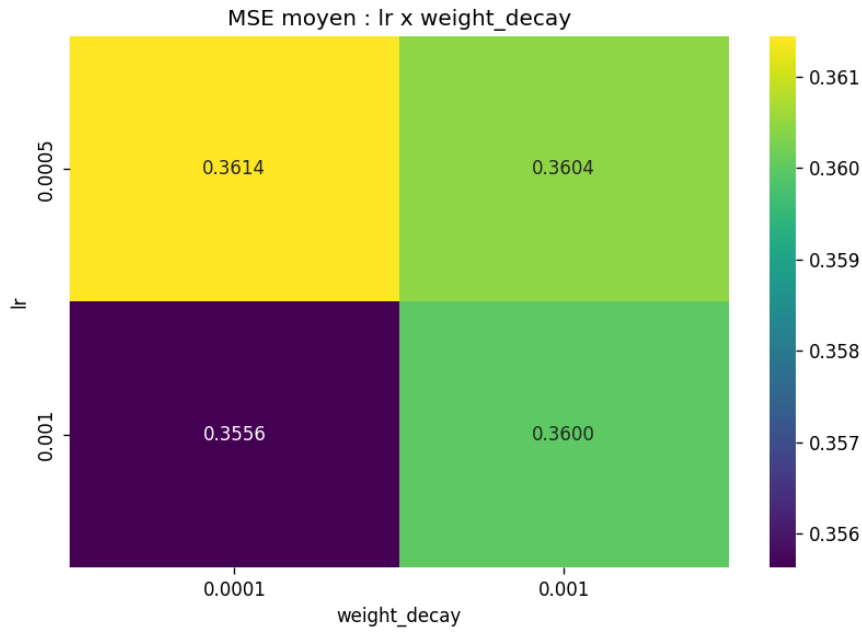


FIGURE 12 – MSE moyenne selon le couple learning rate / weight decay.

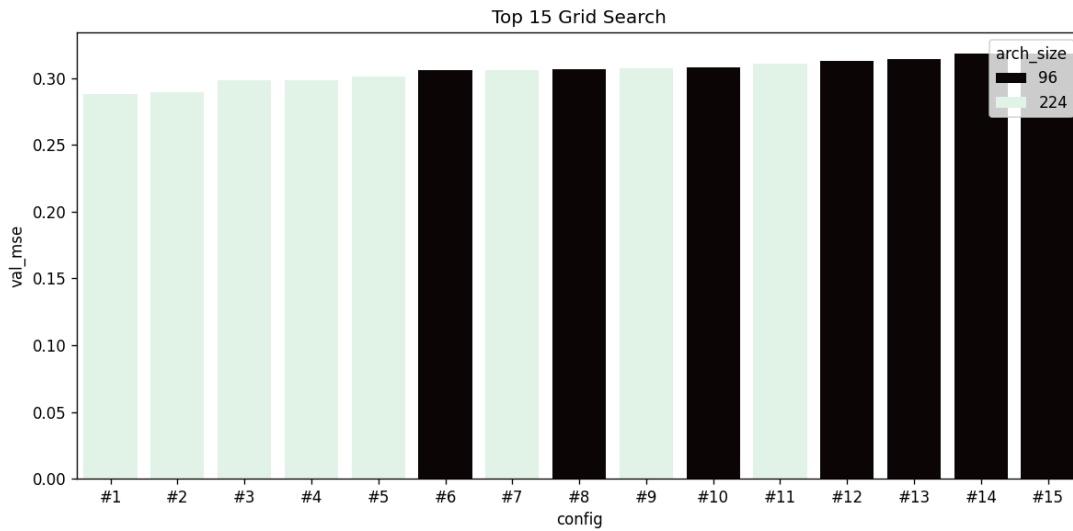


FIGURE 13 – Les 15 meilleures configurations du Grid Search.

6.3 Impact des hyperparamètres

TABLE 4 – Impact individuel estimé par l'écart-type des moyennes de MSE.

Hyperparamètre	Écart-type des moyennes
hidden_dims	0.03239
dropout_rate	0.02068
lr	0.00221
weight_decay	0.00119
activation	0.00090

L'architecture est l'hyperparamètre le plus influent dans le Grid Search. Le taux de Dropout arrive en deuxième position. Le learning rate, le weight decay et l'activation ont un impact plus faible dans la grille réduite testée.

7 Exercice 5 – Random Search

7.1 Espace de recherche

Le Random Search tire les hyperparamètres continus selon des distributions uniformes ou log-uniformes. Le choix log-uniforme pour le learning rate et le weight decay est important, car ces paramètres agissent sur plusieurs ordres de grandeur.

7.2 Résultats obtenus

TABLE 5 – Top 5 des configurations Random Search.

Architecture	Activation	Dropout	LR	Weight decay	Val MSE
[256, 128, 64, 32]	leaky_relu	0.0565	0.002871	0.000483	0.2779
[256, 128, 64, 32]	relu	0.2857	0.003489	0.000012	0.2861
[128, 64, 32]	leaky_relu	0.3242	0.000710	0.002778	0.2913
[128, 64, 32]	relu	0.1644	0.002864	0.000638	0.2938
[128, 64, 32]	leaky_relu	0.2743	0.000824	0.000084	0.2967

La meilleure configuration Random Search obtient une MSE de validation de 0.2779, ce qui est meilleur que la meilleure configuration Grid Search. Elle utilise une architecture plus large [256, 128, 64, 32], LeakyReLU, un faible Dropout et un learning rate plus élevé que les valeurs explorées par la grille.

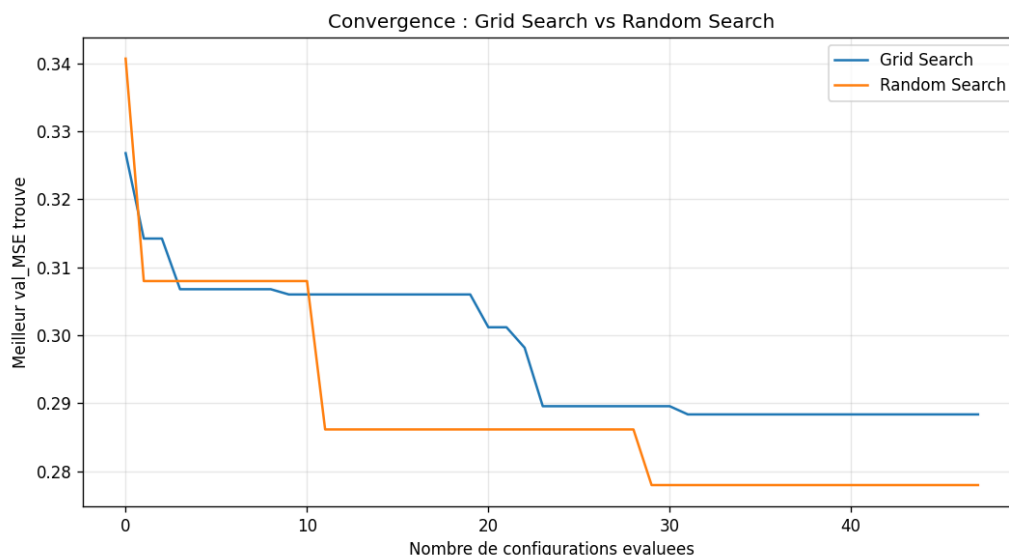


FIGURE 14 – Convergence comparative : Grid Search vs Random Search.

Le Random Search atteint une meilleure performance finale. Son avantage vient du fait qu'il explore des valeurs continues, notamment pour le learning rate et le weight decay,

alors que le Grid Search reste limité à quelques valeurs fixes.

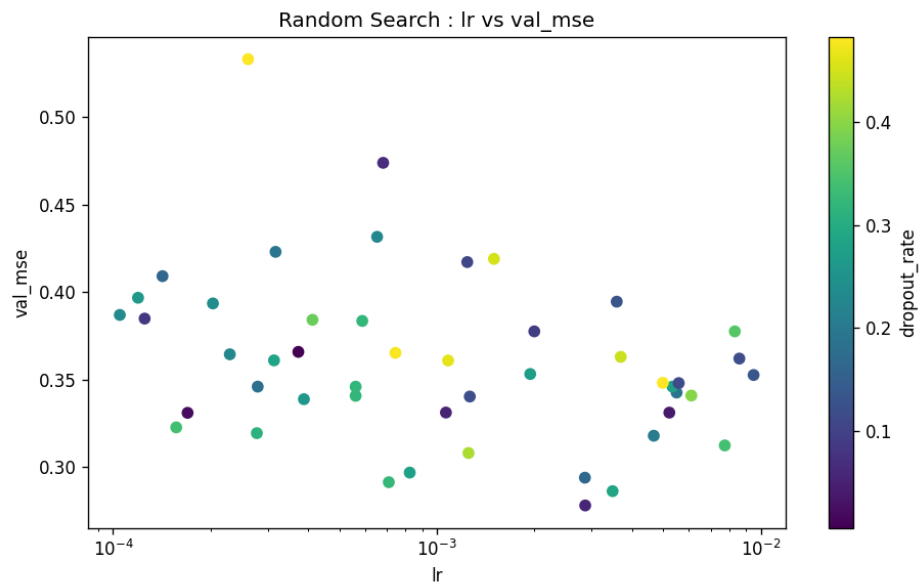


FIGURE 15 – Relation entre learning rate et MSE de validation.

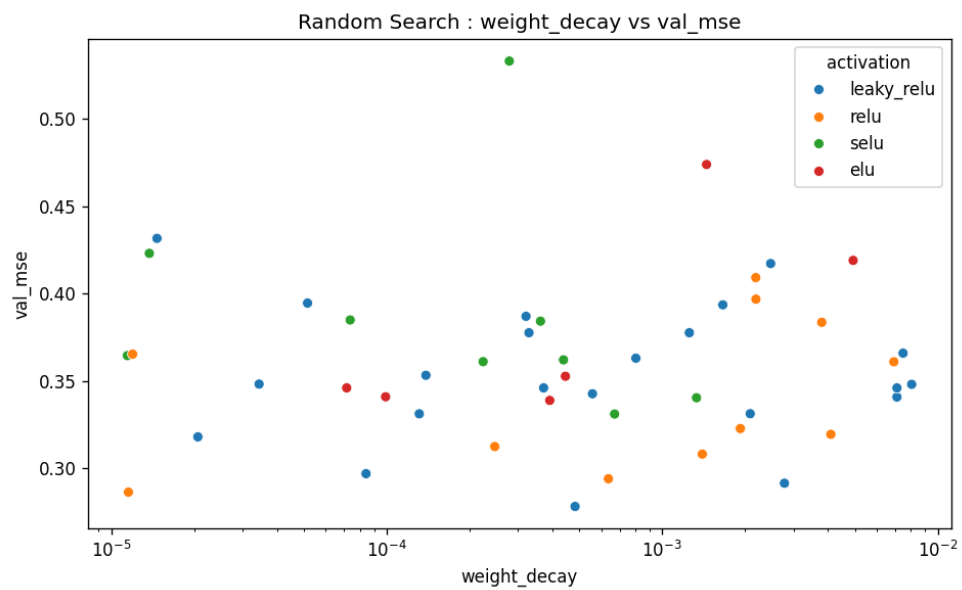


FIGURE 16 – Relation entre weight decay et MSE de validation.

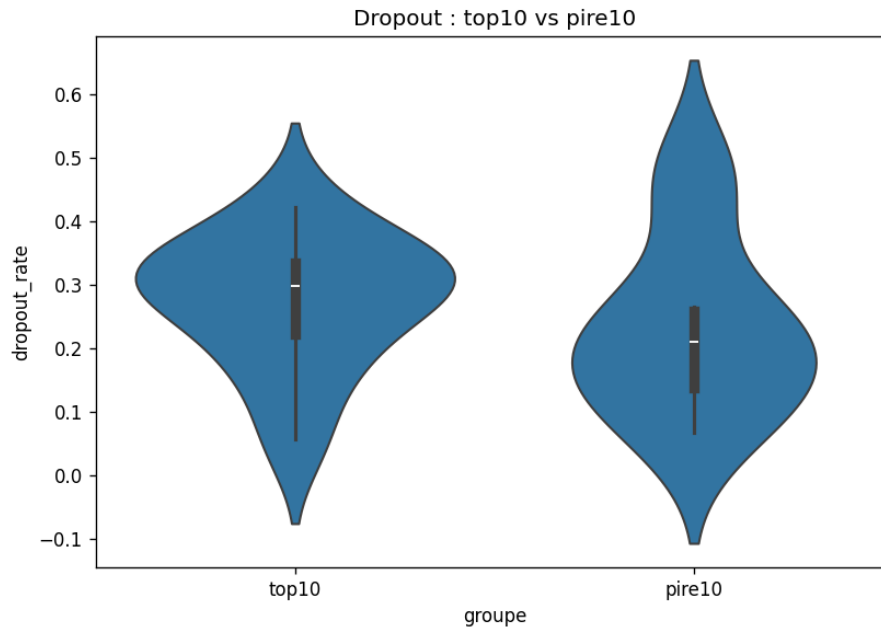


FIGURE 17 – Distribution du Dropout pour les 10 meilleures et les 10 moins bonnes configurations.

7.3 Corrélations de Spearman

TABLE 6 – Corrélations de Spearman entre hyperparamètres continus et MSE de validation.

Hyperparamètre	Corrélations Spearman
lr	-0.3067
clip_value	-0.0907
dropout_rate	-0.0147
weight_decay	-0.0096

La corrélation la plus importante en valeur absolue concerne le learning rate. Dans cet ensemble d'essais, des learning rates plus élevés sont associés à de meilleures MSE de validation, jusqu'à une certaine limite. Le clipping, le Dropout et le weight decay ont des corrélations plus faibles dans l'espace exploré.

8 Exercice 6 – Évaluation finale

8.1 Validation sur le jeu de test

Les meilleures configurations ont été réentraînées sur train+validation. Le jeu de test n'a été utilisé qu'une seule fois pour l'évaluation finale.

TABLE 7 – Résultats finaux sur le jeu de test.

Modèle / Config	Val MSE	Test MSE	Test MAE	Test R^2	Temps (s)
Baseline	—	0.2949	0.3796	0.7772	98.6
Meilleur Grid Search	0.2883	0.2771	0.3665	0.7907	277.0
Random Search #1	0.2779	0.2509	0.3377	0.8105	244.7
Random Search #2	0.2861	0.3217	0.3908	0.7570	83.5
Random Search #3	0.2913	0.2841	0.3594	0.7854	204.4

Le meilleur modèle final est **Random Search #1**. Il obtient un Test MSE de 0.2509, un MAE de 0.3377 et un R^2 de 0.8105. Il généralise mieux que la baseline et que le meilleur modèle du Grid Search.

8.2 Analyse des erreurs

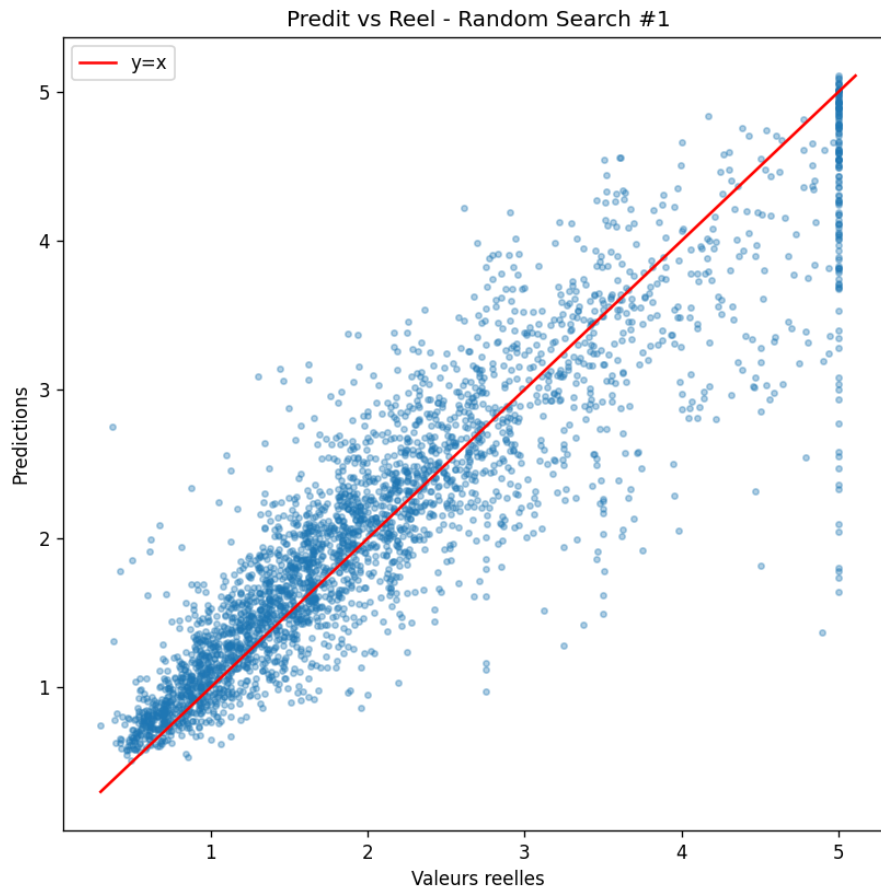


FIGURE 18 – Comparaison entre valeurs réelles et valeurs prédites sur le test set.

Le nuage de points suit globalement la droite $y = x$, ce qui indique une bonne capacité prédictive. Cependant, les valeurs proches du plafond de 5.0 restent plus difficiles à prédire, ce qui est cohérent avec le plafonnement artificiel observé dans la cible.

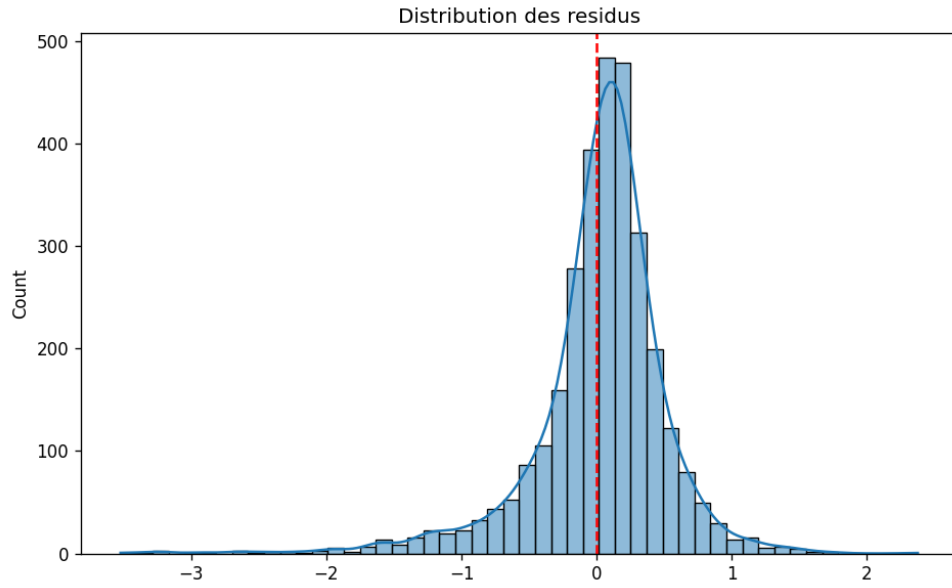


FIGURE 19 – Distribution des résidus $y_{pred} - y_{true}$.

Le test de Shapiro-Wilk donne une statistique de 0.8877 et une p-value de 8.829×10^{-43} . On rejette donc l'hypothèse de normalité des résidus. Les résidus ne suivent pas parfaitement une loi normale, ce qui est courant dans un problème de régression tabulaire avec cible plafonnée et structure géographique.

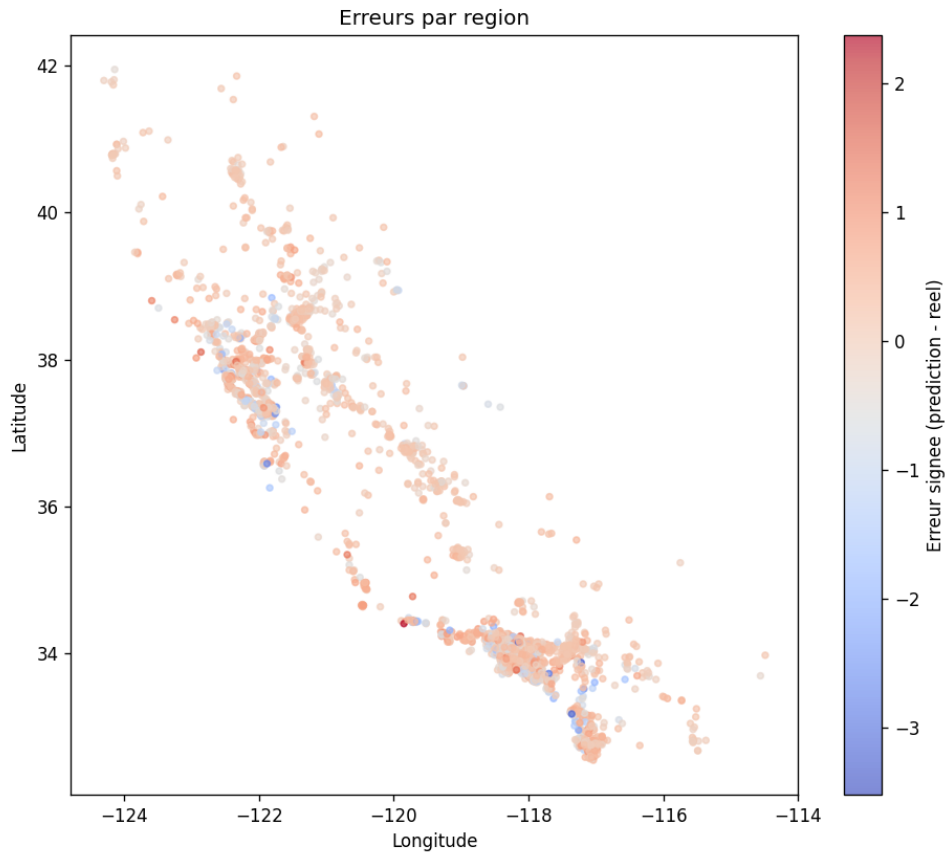


FIGURE 20 – Carte des erreurs signées selon la longitude et la latitude.

L’analyse spatiale montre que les erreurs ne sont pas uniformes géographiquement. Les régions où le marché immobilier est plus spécifique ou plus coûteux, notamment certaines zones côtières et urbaines, peuvent être plus difficiles à modéliser avec un simple MLP utilisant uniquement les variables disponibles.

9 Synthèse et discussion

Les expériences confirment l’importance du prétraitement, notamment la standardisation des variables. Sans normalisation, les gradients deviennent plus instables, et le gradient clipping devient plus important pour sécuriser l’apprentissage. BatchNorm, Dropout et L2 sont des techniques utiles, mais leur effet dépend du dataset, de l’architecture et de l’intensité de régularisation choisie. Dans nos résultats, la baseline complète est parfois dépassée par des variantes moins régularisées, ce qui suggère que le modèle était potentiellement trop contraint.

Le Grid Search est simple à interpréter et permet une comparaison méthodique des combinaisons discrètes. Cependant, il devient rapidement coûteux : la grille complète comportait 288 configurations, alors que la grille réduite en contenait 48. Le Random Search s’est montré plus efficace dans ce TP, car il a découvert une configuration avec un learning rate et un Dropout qui n’étaient pas exactement présents dans la grille.

La meilleure performance finale est obtenue par Random Search #1 avec un R^2 de 0.8105 sur le test set. Cette performance reste perfectible. Les limites principales sont le plafonnement de la cible, l’absence de validation croisée complète pour les expériences coûteuses et la simplicité du MLP pour capturer des effets spatiaux complexes. Des améliorations possibles seraient l’optimisation bayésienne, Hyperband, ASHA, des ensembles de modèles, ou encore l’ajout de features géographiques plus riches.

10 Conclusion

Ce TP a permis de mettre en oeuvre un pipeline complet de Deep Learning pour une tâche de régression réelle : exploration des données, préparation stricte sans fuite de données, implémentation d’un MLP profond, stabilisation de l’entraînement, recherche d’hyperparamètres et évaluation finale. Les résultats montrent que la recherche d’hyperparamètres a un impact majeur sur la performance. Le Random Search a fourni la meilleure configuration et a amélioré le modèle baseline. Le test set a été réservé à l’évaluation finale, conformément aux bonnes pratiques expérimentales.